

Section 4: KV-Cache Optimization

Literature Review on LLM Context Management

J.C. Vaught

January 27, 2026

1 Introduction to KV-Cache Memory Bottleneck

The Key-Value (KV) cache represents a critical memory bottleneck in large language model (LLM) inference, growing linearly with both sequence length and batch size. During autoregressive generation, the KV cache stores attention keys and values for all previously processed tokens, requiring substantial GPU memory. For instance, a KV-cache representing a 128k token context window for a single user (batch size 1) consumes approximately 40 GB of memory with LLaMA 3 70B models. The cache size follows the formula: $2 \cdot n \cdot h \cdot d \cdot e \cdot b \cdot l$, where n is the number of layers, h is the number of heads, d is the dimension, e is element size, b is batch size, and l is sequence length.

1.1 Memory Bandwidth Constraints

LLM inference is fundamentally memory bandwidth-bound rather than compute-bound. Generating each new token requires repeatedly loading and storing the entire KV cache, creating substantial slowdown. Existing systems exhibit only 20-38% utilization of allocated KV cache memory, with 60-80% wasted due to fragmentation and over-reservation. This inefficiency motivates the development of various optimization strategies including eviction policies, quantization techniques, architectural modifications, and memory management innovations.

2 KV-Cache Eviction Policies

Eviction policies selectively discard or compress KV cache entries to maintain manageable memory footprints while preserving model performance. These methods operate on the principle that not all cached tokens contribute equally to attention computation.

2.1 Importance-Based Eviction

2.1.1 H2O: Heavy-Hitter Oracle

Zhang et al. (2023) introduced H₂O (Heavy-Hitter Oracle), an eviction policy based on the observation that a small portion of tokens (“heavy hitters”) contributes most of the value when computing attention scores. These heavy hitters strongly correlate with frequent token co-occurrence in text, and their removal results in significant performance degradation. H₂O dynamically retains a balance of recent tokens and heavy hitters, achieving up to 29× throughput improvement over systems like DeepSpeed Zero-Inference and HuggingFace Accelerate, and up to 1.9× latency reduction on OPT-6.7B and OPT-30B models. With only 20% of tokens retained as heavy hitters, H₂O maintains model quality while substantially reducing memory requirements.

Citation: Zhang, Z., et al. (2023). H_2O : Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. NeurIPS 2023. arXiv:2306.14048

2.1.2 ScissorHands

ScissorHands (Liu et al., 2023) maintains KV cache memory at a fixed budget without model fine-tuning, based on the “persistence of importance” hypothesis: pivotal tokens that substantially influenced one generation step will significantly influence future generations. The system manages the cache by storing pivotal tokens with higher probability, achieving up to $5\times$ reduction in KV cache memory usage without compromising model quality. When combined with 4-bit quantization, ScissorHands achieves up to $20\times$ compression. The method assumes cache importance stability, where a small set of entries remains consistently important throughout generation.

Citation: Liu, Z., et al. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. NeurIPS 2023. arXiv:2305.17118

2.2 Attention Sink and Streaming Methods

2.2.1 StreamingLLM

Xiao et al. (2024) discovered the “attention sink” phenomenon in their StreamingLLM framework: initial tokens receive strong attention scores even when semantically unimportant, serving as a sink for attention weights. Keeping just the KV states of these initial tokens (as few as 4 tokens) together with a sliding window of recent tokens enables LLMs to generalize to infinite sequence lengths without fine-tuning. StreamingLLM enables models like LLaMA-2, MPT, Falcon, and Pythia to perform stable language modeling with up to 4 million tokens, achieving up to $22.2\times$ speedup over sliding window recomputation baselines. The method has been integrated into production systems including NVIDIA TensorRT-LLM and HuggingFace Transformers.

Citation: Xiao, G., et al. (2024). Efficient Streaming Language Models with Attention Sinks. ICLR 2024. arXiv:2309.17453

2.3 Adaptive and Layer-Aware Eviction

2.3.1 FastGen

FastGen (Ge et al., 2024) employs targeted attention profiling to discern the intrinsic structure of attention modules, then constructs KV cache adaptively across different attention patterns. The method evicts long-range contexts on attention heads emphasizing local contexts, discards non-special tokens on attention heads centered on special tokens, and employs standard KV cache only for heads that broadly attend to all tokens. This adaptive approach achieves up to 55% latency reduction at 16k generation length and 44.9% pruned ratio on LLaMA 1-65B, with negligible generation quality loss. FastGen operates as a plug-and-play method requiring no resource-intensive fine-tuning.

Citation: Ge, S., et al. (2024). Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. ICLR 2024. arXiv:2310.01801

2.3.2 PyramidKV and SnapKV

Li et al. (2024) introduced SnapKV, which selects a fixed number of tokens per attention head by retaining those with highest estimated contribution to the attention distribution based on an observation window. Cai et al. (2024) extended this with PyramidKV, which assigns layer-specific

budgets with larger allocations to lower layers, based on the principle that deeper layers require less KV cache. PyramidKV preserves performance using just 12% of full KV cache (2048 tokens) on LongBench and retains quality with only 0.7% cache in extreme conditions. Both methods estimate token importance using total attention received from the last 64 input tokens, smoothed with a moving average.

Citations: Li, Y., et al. (2024). SnapKV: Efficient KV Cache Compression. arXiv preprint. / Cai, Z., et al. (2024). PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. arXiv:2406.02069

3 KV-Cache Quantization

Quantization reduces the precision of stored KV cache entries, directly decreasing memory footprint and memory bandwidth requirements. The challenge lies in maintaining attention accuracy despite reduced numerical precision.

3.1 Per-Channel and Per-Token Quantization

3.1.1 KIVI

Liu et al. (2024) developed KIVI, a tuning-free asymmetric 2-bit quantization method for KV cache presented at ICML 2024. KIVI’s key insight is that keys should be quantized per-channel (grouping elements along the channel dimension), while values should be quantized per-token. This asymmetric approach enables LLaMA, Falcon, and Mistral models to maintain nearly identical quality while using $2.6\times$ less peak memory (including model weights), supporting up to $4\times$ larger batch sizes and achieving $2.35\text{--}3.47\times$ throughput improvements on real workloads. KIVI operates as a plug-and-play solution requiring no fine-tuning.

Citation: Liu, Z., Yuan, J., et al. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. ICML 2024. arXiv:2402.02750

3.1.2 KVQuant

Hooper et al. (2024) introduced KVQuant, presented at NeurIPS 2024, targeting extremely long context lengths up to 10 million tokens. KVQuant incorporates four novel techniques: (1) Per-Channel Key Quantization, adjusting quantization dimensions to match key activation distributions; (2) Pre-RoPE Key Quantization, quantizing before applying rotary positional embeddings; (3) Non-Uniform KV Cache Quantization, deriving per-layer sensitivity-weighted datatypes; and (4) Per-Vector Dense-and-Sparse Quantization, isolating outliers per vector to minimize quantization range skew. KVQuant achieves less than 0.1 perplexity degradation with 3-bit quantization on WikiText-2 and C4, enabling inference of LLaMA-7B with 1M context on a single A100 GPU and 10M context on an 8-GPU system with $8\times$ KV cache compression.

Citation: Hooper, C., et al. (2024). KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. NeurIPS 2024. arXiv:2401.18079

3.2 Hybrid Quantization Approaches

3.2.1 GEAR

Kang et al. (2024) proposed GEAR, an efficient KV cache compression framework achieving near-lossless high-ratio compression through a three-pronged approach: (1) ultra-low-precision (4-bit)

quantization for the majority of similarly-scaled entries, (2) low-rank matrix approximation via SVD to capture quantization residuals, and (3) sparse matrix correction for individual outlier errors. This hybrid approach addresses the challenge that autoregressive decoding compounds errors at each step. GEAR achieves near-lossless 4-bit compression with up to $2.38\times$ throughput improvement and $2.29\times$ peak memory reduction.

Citation: Kang, M., Zhang, X., et al. (2024). GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. arXiv:2403.05527

4 Architectural Modifications

Architectural changes to the transformer attention mechanism can fundamentally reduce KV cache requirements by sharing or reducing the number of key-value heads.

4.1 Multi-Query and Grouped-Query Attention

4.1.1 Multi-Query Attention (MQA)

Shazeer (2019) introduced Multi-Query Attention (MQA) in “Fast Transformer Decoding: One Write-Head is All You Need.” MQA modifies standard Multi-Head Attention (MHA) by sharing a single key-value head across all query heads, while maintaining separate query projections. Shazeer’s analysis identified memory bandwidth for loading KV cache as the primary performance limiter on modern accelerators (GPUs, TPUs). MQA dramatically reduces KV cache size proportional to the number of attention heads, improving inference efficiency with minimal accuracy degradation.

Citation: Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150

4.1.2 Grouped-Query Attention (GQA)

Ainslie et al. (2023) proposed Grouped-Query Attention (GQA) as a generalization interpolating between MHA and MQA. GQA divides query heads into groups, with each group sharing a single key-value head. This design captures MQA’s efficiency benefits while mitigating quality trade-offs. GQA is formally equivalent to MHA when the number of groups equals the number of heads, and equivalent to MQA with a single group. Following publication in May 2023, GQA was rapidly adopted by production models including Meta’s LLaMA 2 (July 2023) and LLaMA 3 (2024), demonstrating its practical effectiveness.

Citation: Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245

4.2 Cross-Layer KV Sharing

4.2.1 Cross-Layer Attention (CLA)

MIT researchers (2024) introduced Cross-Layer Attention (CLA), which enables sharing of key-value heads not only within layers but also across adjacent layers. CLA computes key-value projections for only a subset of layers, allowing subsequent layers to reuse KV activations from previous layers. At both 1B and 3B model scales, applying CLA with a sharing factor of 2 (CLA2) yields $2\times$ KV cache reduction while incurring less than 1% perplexity degradation for Multi-Query Attention

(MQA) models with typical head sizes of 64 and 128. CLA is orthogonal to both MQA and GQA, enabling combination with either technique.

Citation: Brandon, W., et al. (2024). Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. arXiv:2405.12981

4.2.2 Layer-Condensed KV Cache (LCKV)

Yang et al. (2024) systematically investigated cross-layer KV sharing techniques in their work accepted to ACL 2024. Their unified framework covers multiple recent methods and novel variants, uniformly dividing transformer layers into groups of adjacent layers and pairing queries of all layers in each group with KVs from the bottom layer. When reducing KV cache size by $2\times$, most configurations achieve higher throughput than standard transformers while maintaining competitive performance. A systematic study by Tu et al. (2024), accepted to NAACL 2025, further demonstrated that cross-layer sharing enables $2\times$ throughput improvements while preserving model quality.

Citations: Yang, Y., et al. (2024). Layer-Condensed KV Cache for Efficient Inference of Large Language Models. ACL 2024. / Tu, K., et al. (2024). A Systematic Study of Cross-Layer KV Sharing for Efficient LLM Inference. NAACL 2025. arXiv:2410.14442

4.3 Depth-Dimension Compression

4.3.1 MiniCache

Jia et al. (2024) developed MiniCache, presented at NeurIPS 2024, which compresses KV cache across the depth (layer) dimension by exploiting high similarity between adjacent layers in middle-to-deep portions of LLMs. MiniCache disentangles KV states into magnitude and direction components, interpolating directions while preserving magnitudes. The method also introduces token retention to keep highly distinct state pairs unmerged, minimizing information loss. On ShareGPT, LLaMA-2-7B with 4-bit MiniCache achieves $5.02\times$ compression, approximately $5\times$ throughput enhancement, and 41% memory footprint reduction compared to FP16 full cache baseline.

Citation: Jia, Z., et al. (2024). MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. NeurIPS 2024. arXiv:2405.14366

5 Memory Management Innovations

Beyond reducing cache size, innovative memory management strategies address fragmentation and enable efficient sharing of KV cache across requests.

5.1 PagedAttention and vLLM

Kwon et al. (2023) introduced PagedAttention, inspired by virtual memory and paging in operating systems, presented at SOSP 2023. Traditional attention algorithms require contiguous memory for keys and values, but PagedAttention partitions KV cache into blocks (analogous to memory pages), with each block containing keys and values for a fixed number of tokens. Contiguous logical blocks map to non-contiguous physical blocks via block tables, enabling flexible memory allocation. Memory waste occurs only in the last block of each sequence, achieving near-optimal utilization with under 4% waste compared to 60-80% waste in existing systems due to fragmentation and over-reservation. The vLLM serving system built on PagedAttention improves throughput by $2-4\times$ compared to FasterTransformer and Orca while maintaining equivalent latency.

Citation: Kwon, W., Li, Z., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180

6 Conclusion and Future Directions

KV-cache optimization represents a critical frontier in LLM inference efficiency, with solutions spanning eviction policies, quantization, architectural modifications, and memory management. Recent advances demonstrate that substantial memory reductions (2-20 $\times$) are achievable with minimal quality degradation through careful application of these techniques. Future research directions include:

- **Unified frameworks** combining complementary techniques (e.g., quantization with eviction)
- **Hardware-aware optimization** tailored to specific accelerator architectures
- **Dynamic adaptation** adjusting compression strategies based on input characteristics
- **Extreme context lengths** pushing toward 100M+ token contexts
- **Multi-modal KV cache** extending techniques to vision-language models

The rapid pace of innovation in this area, particularly throughout 2023-2024, reflects both the critical importance of KV cache management and the substantial headroom for further optimization. As model sizes and context requirements continue to grow, KV-cache optimization will remain essential for practical LLM deployment.